AgWeatherNet AI Chatbot

Cpts 421

Presented by Team 15

TABLE OF CONTENTS

1. Introduction
2. Sprint Objectives
3. Feature Implementation
4. Prototype & Client Feedback
5. Kanban Overview & Team Contributions
6. Documentations
7. Responsible Use of AI
8. Sprint Achievements and Challenges
9. Next Steps and Sprint Retrospective
10. Conclusion

INTRODUCTION

- **Project Introduction:**
  - AI Chatbot for AgWeatherNet Data Access
  - Focus on usability and accurate data retrieval
- **Team Roles:**
  - Clearly defined roles across development, research, and integration
  - Responsibilities include backend development, data handling, and frontend design
- **Sprint 2 Goals:**
  - Implement the RAG pipeline foundation
  - Integrate the embedding model with the vector database
  - Deliver a functional end-to-end prototype connecting the UI, language model, and AgWeatherNet data pipeline

SPRINT OBJECTIVES

- **Defined Objectives:**
  - Determine System Architecture
  - Prototype RAG Pipeline
  - Chatbot User Interface
- **Key Goals and Rationale:**
  - Establish a clear system structure to guide development and integration
  - Validate that the RAG pipeline can retrieve and generate accurate responses
- **Connection to Overall Vision:**
  - Provides a foundation for building a scalable and user-friendly AgWeatherNet chatbot
  - Moves the project toward a functional end-to-end chatbot system
  - Ensures the backend (RAG) and frontend (UI) are aligned from the start

FEATURE IMPLEMENTATION

Chatbot UI Prototype

React + TypeScript + Vite chat interface with sidebar, transcript, and composer

BEFORE & AFTER

Before Sprint 2

- No working backend or chat interface
- Architecture and data flow undefined
- No connection to AgWeatherNet data
- No LLM or embedding integration

After Sprint 2

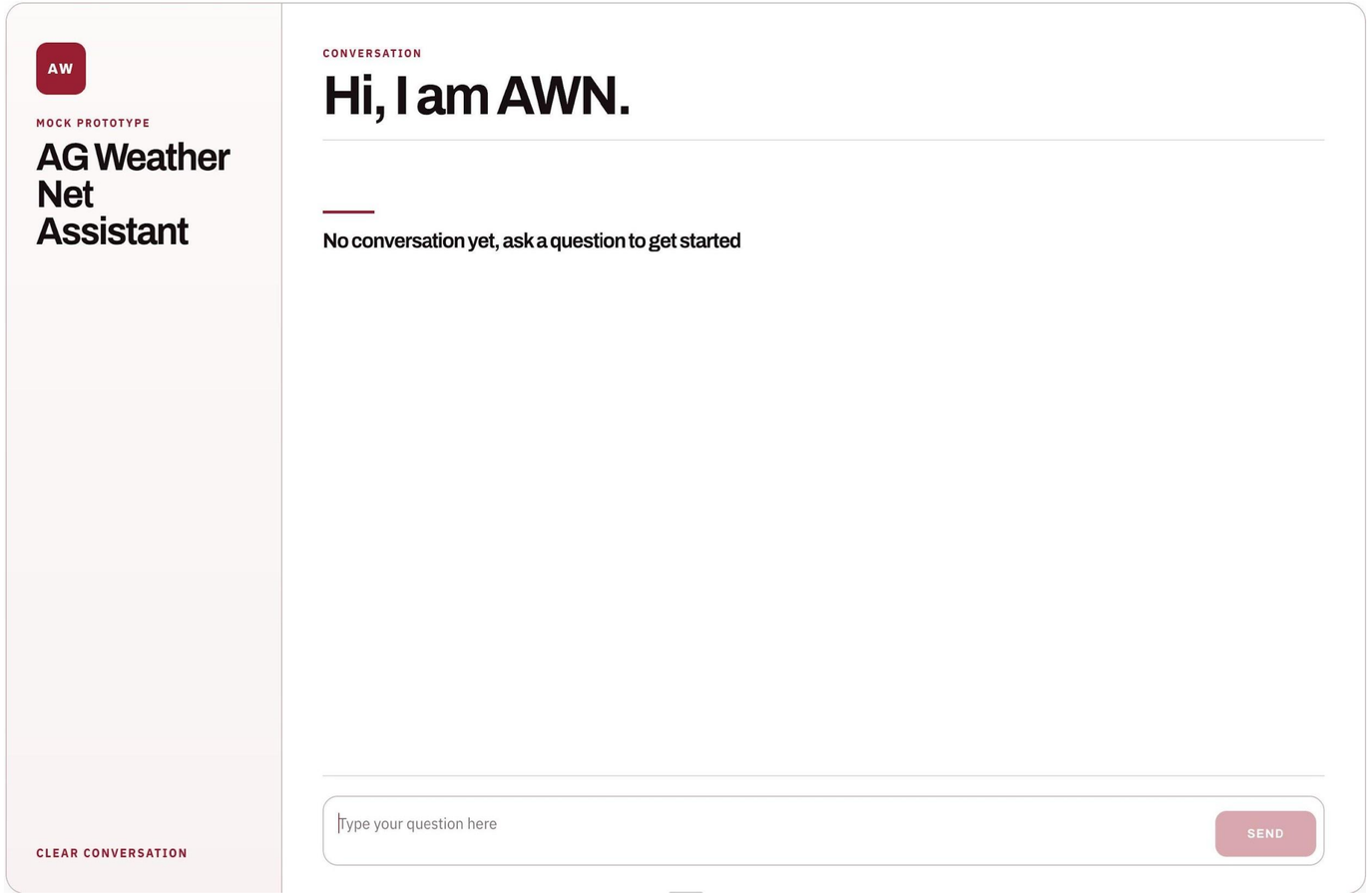
- Functional React chat UI with local message state
- RAG architecture defined and documented
- Read-only MySQL connection to AWN schema
- OpenRouter LLM and embedding wrappers integrated

Pipeline

Current

Next steps before demo

FRONTEND IMPLEMENTATION



AgWeatherNet Assistant — current chat interface mock

PROTOTYPE OVERVIEW

Collapsible chat panel designed to integrate with the existing AgWeatherNet web portal. Core elements:

- Branded sidebar with AgWeatherNet identity
- Scrollable conversation panel
- Auto-resizing composer (Enter sends, Shift+Enter newlines)
- Clear-conversation control for fresh sessions

Backend Wiring

Frontend ↔ Backend Wiring

Vite proxy forwards /api/chat to FastAPI; live end-to-end chat path with loading states

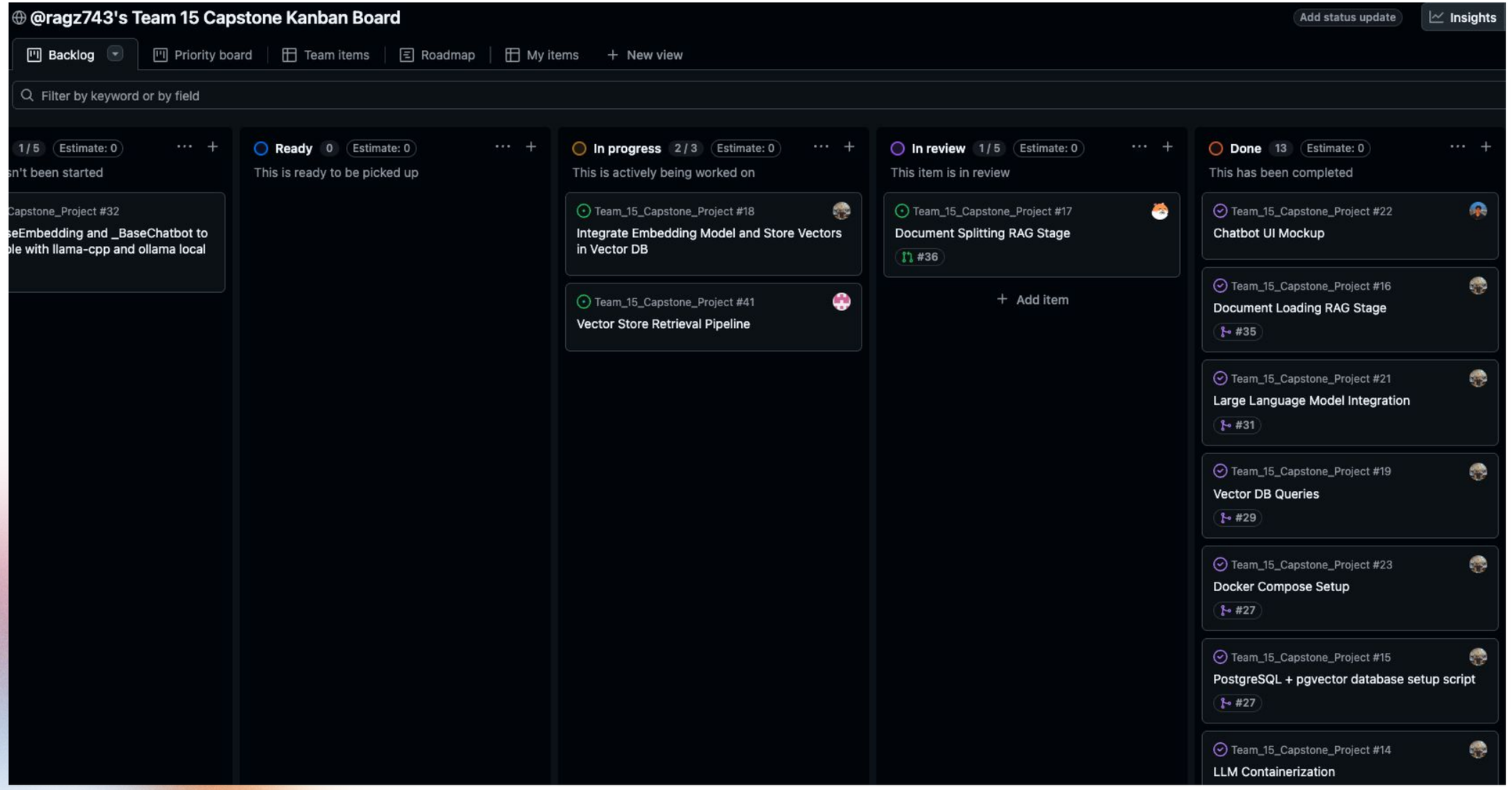
OpenRouter LLM Integration

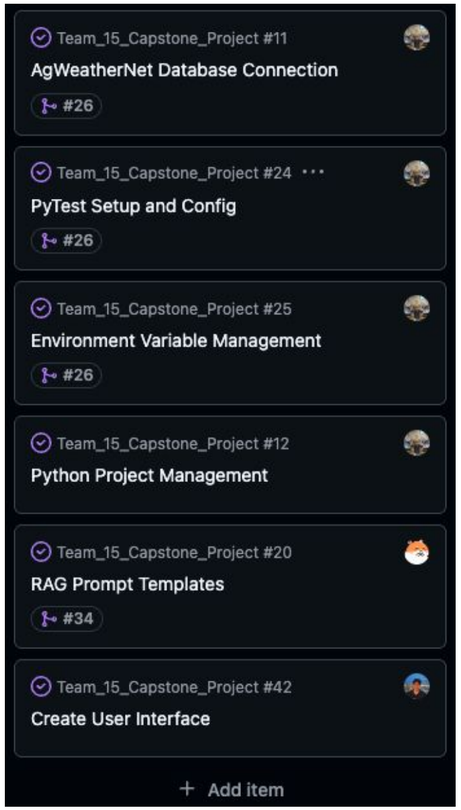
FastAPI delegates to LangChain ChatOpenRouter wrapper; API key kept server-side only

Client Feedback:

Client liked the frontend mock up, no negative feedback

KANBAN OVERVIEW





TEAM CONTRIBUTION

Sai

- Reviewed and tested teammates' pull requests prior to merge
- Communication with the client and professor
- Handled report writings and submissions

Zuriel

- Built a functional React chat UI for the AWN assistant
- Wired the React frontend to the FastAPI backend
- Added backend chat and health endpoints using FastAPI
- Integrated OpenRouter as the LLM provider for chatbot responses

Danbi

- Chatbot LLM Prompt Template classes
- Document Splitting / Processing code
- Reviewed and tested teammates' pull requests prior to merge
- Formatted reports for clarity and consistency
- Handled report submissions

Gavin

- Docker Compose Setup
- PostgresSQL + pgvector database configuration and containerization
- LLM Containerization
- AgWeatherNet + PostgresSQL database connector classes
- Python Project Management, linters, formatters, pytest config, and package management
- Environment variable config
- RAG Component abstract classes + interface
- OpenRouter RAG component wrappers and YAML configuration

SYSTEM ARCHITECTURE

- **Overview:**
  - The system uses a Retrieval-Augmented Generation (RAG) architecture designed to connect LLMs with AgWeatherNet’s weather database.
- **Major Components:**
  - It consists of a React Frontend, a Backend API, and a Semantic Search service using Vector Store indexes (Pgvector).
- **Data Flow:**
  - User natural language queries are processed by the backend, which identifies the necessary operations and retrieves relevant schema or data to generate conversational responses.
- **Tech Choice:**
  - AWS EC2 was selected for deployment to ensure scalability and seamless integration with the existing AgWeatherNet web portal infrastructure. Python and JavaScript (React) were chosen to match the client’s existing technical environment for long-term maintainability.

USER INTERFACE DESIGN

- **Structure:**
  - The UI is built as an interactive chat window integrated directly into the AgWeatherNet web portal using React.
- **Key Features:**
  - It includes a text input box for natural language queries and a response history panel so users can reference previous parts of a conversation.
- **Usability & Feedback:**
  - The design includes context awareness to remember recent station or timeframe details, reducing the need for repeated user inputs based on early usability considerations.
- **Visualization:**
  - The interface is designed to support the display of tables, plots, or charts within the chat window to help users visualize weather trends.

TESTING PLAN

- **Unit Testing:**
  - Validates individual modules like query processing, API handling, and response generation.
- **Integration Testing:**
  - Verifies stable communication between the chatbot and the AgWeatherNet backend APIs
  - Also, verifies stable communication between the backend, pgvector store, and AgWeatherNet MySQL
- **System Testing:**
  - Uses simulated real-world queries to confirm overall accuracy and relevance of the AI chatbot’s responses.

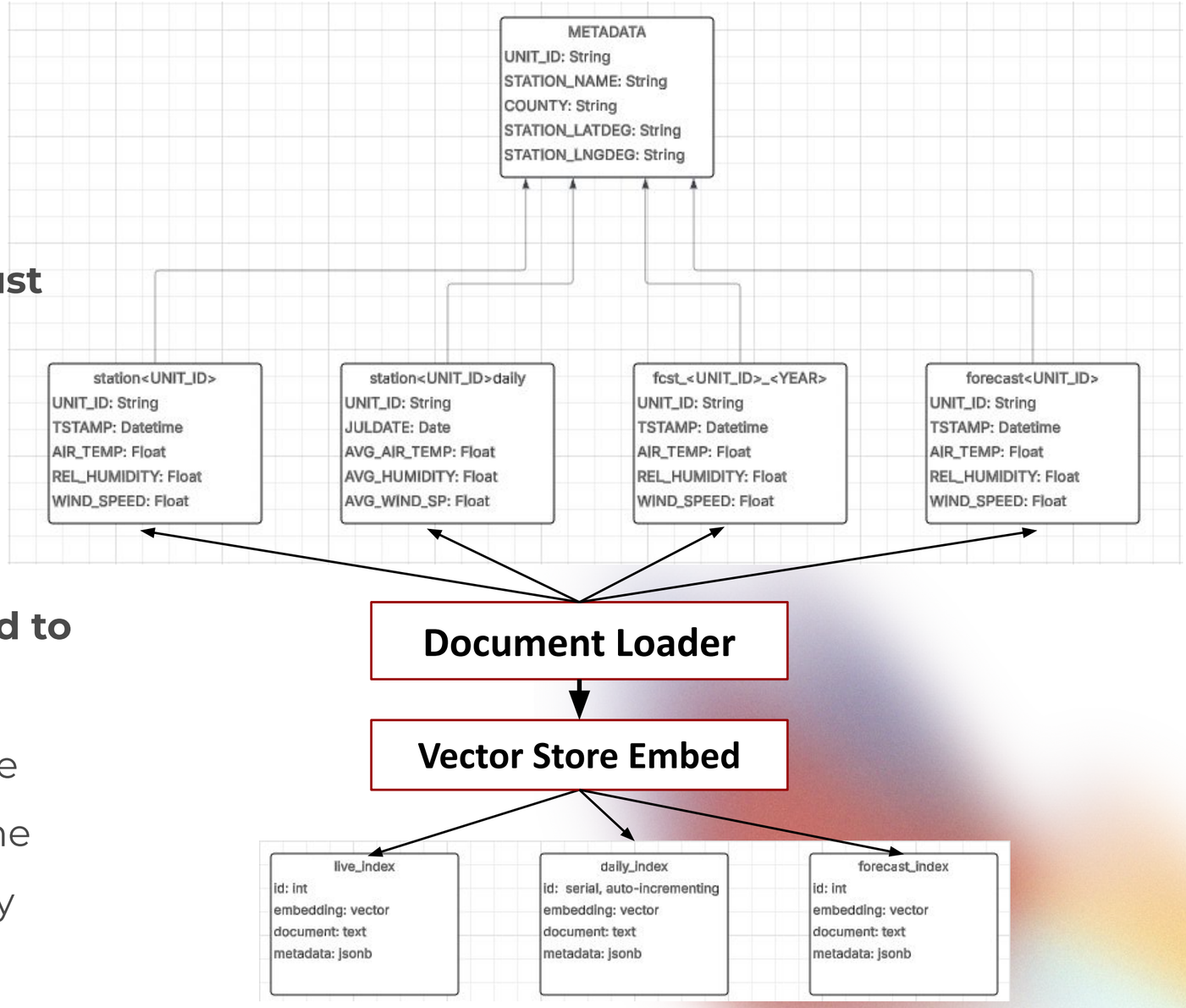
TESTING PLAN

- **Tools & Methods**:
  - A dedicated testing suite was developed to quantify system correctness and measure response times, ensuring that they stay under the 5-second average threshold.
- **Issue Resolution**:
  - Guardrails were implemented to handle “failure modes” such as ambiguous station names, missing data, or out-of-scope prompts
- **CI Pipeline (Planned):**
  - GitHub Actions to run pytest automatically on every PR, catching regressions in accuracy and performance

DATABASE DESIGN

- **Two Primary Components:**
  - AWN MySQL Weather Database
  - New PostgreSQL pgvector Database
- **For relevant stations the RAG pipeline must read and store:**
  - current conditions
  - historical conditions
  - forecasted conditions
- **After processing, embeddings are indexed to be searched later:**
  - Note: daily (historical index) keeps unique records for summaries of each day. On the other hand, live and forecast indexes only store most recent entries.

Retrieving Data From AWN



Storing in the PostgreSQL vector store

RESPONSIBLE USE OF AI

Disclosure:

GenAI tools such as ChatGPT were used during this sprint to assist with certain documentation tasks

RESPONSIBLE USE OF AI

- **Application & Improvements:**
  - AI tools were used to help draft documentation and support brainstorming ideas
  - This improved efficiency and reduced the time spent on initial drafting
- **Limitations & Checks:**
  - Some AI-generated outputs required refinement or correction
  - The team reviewed all AI-assisted content to ensure accuracy, relevance, and alignment with project goals
- **Accountability:**
  - All final documentation, design decisions, and deliverables were reviewed and approved by the team
  - The team maintains full responsibility for the accuracy and quality of the final work

SPRINT 2 ACHIEVEMENTS

- **Achievements:**
  - Completed Requirements Specification and Solution Approach reports (system architecture defined)
  - Established a working baseline prototype
  - Developed a basic UI for the prototype
  - Evaluated and selected architecture based on constraints (e.g., memory cost, scalability)

SPRINT 2 CHALLENGES

- **Challenges and Solutions:**
  - Multiple architecture options with trade-offs → Narrowed down by prioritizing efficiency and feasibility
  - Balancing performance with resource constraints → Chose a lightweight approach for the prototype
- **Progress from Sprint 1:**
  - Moved from initial concept to a defined system architecture
  - Advanced from planning to a functional prototype with UI
  - Improved clarity on technical direction and implementation strategy

NEXT STEPS

- **Upcoming Tasks:**
  - Finalize prototype for client demo (Front-end and back-end integration)
  - Refine chatbot UI for demo readiness and usability
- **Sprint Retrospective:**
  - Task distribution needs improvement for better efficiency
  - Presentations for the client were well-prepared and effective
- **Client Feedback Consideration:**
  - Addressed the request for architecture specification through the Solution Approach report
  - Conducted research and provided vectorDB size estimation to align with the client's budget constraints

CONCLUSION

- **Summary:**
  - Defined system architecture and completed key documentation (Requirements Specification, Solution Approach)
  - Developed a baseline chatbot prototype with a basic UI
  - Implemented and evaluated a RAG pipeline approach for accurate data retrieval
  - Progressed toward a client-ready demo with refined functionality and design direction

THANK YOU